

STANCE: Stiffness and Torque Adaptation for Non-linear Compliant Environments

Group 42: Vishal Mangla Mukul Yadav, Muthaiah Mahadevan

Why (Problem and Proposed Solution)

Most passive lower-limb prostheses use a fixed-stiffness ankle where a prosthetist sets its compliance once, and it stays the same regardless of terrain. This works fine on flat, rigid, predictable ground, where that single setting can be tuned in advance.

Powered prostheses improve on this by adjusting for walking speed and different rigid terrains. But no existing control strategy adjusts for compliant ground, where the surface itself gives way unpredictably underfoot, because the ankle can't be pre-set for mechanics it won't know until the moment it lands. As a consequence, more than half of lower-limb prosthesis users fall at least once a year, many during "weight acceptance": the 100–200 ms window after heel strike when full body weight shifts onto the prosthetic leg. In that window, the ankle must get its stiffness right immediately. Too stiff, and it slams the impact into the residual limb, causing pain. Too compliant, and the leg simply folds.

Which failure occurs isn't up to the user. It's decided by the ground. Concrete, wet grass, gravel, and sand compress, damp, and grip differently, and some stay dented afterward, so a stiffness that's safe on one surface can fail on the next. No commercial sensor can measure ground compliance before contact, and vision doesn't help either. We aim to train a single agent that controls a powered ankle by setting three things at once: target angle, stiffness, and damping. Moreover, we'll use only real prosthesis signals: encoder, motor current, load cells, and IMU.

Why Reinforcement Learning

Method	Why it fails here	What RL does instead
FSM impedance control (clinical standard)	Stiffness is fixed in advance for a listed set of situations. The surface about to be landed on is not on that list.	Learning continuous mapping from sensor history to stiffness, with no list of surfaces to enumerate.
Online estimation + adaptive control	Needs force data, that only appears <i>after</i> impact begins, to estimate stiffness.	Sets stiffness at touchdown from what it learned in training, then refines it as force builds — rather than waiting for a measurement that arrives too late to use.
Model Predictive Control	Requires a contact model. No fixed parameter set describes stochastic ground deformation. Solving a contact problem at 200 Hz on a prosthesis is unrealistic.	Moves the optimization offline into training. Inference is a fixed, small cost per step.

The system is heavily underactuated. Five degrees of freedom, one motor. No mathematical inverse exists from a desired body motion to an ankle torque. This is structural, not a matter of difficulty.

The ground cannot be memorized. Properties are drawn from continuous ranges, re-drawn every episode, and the surface dents permanently. The ground's effective stiffness depends on how the agent itself has been loading it. There is no fixed environment parameter to memorize.

Problem Model

The mechanism (5 degrees of freedom, 1 motor): A 0.26 m foot is free to translate and rotate in the plane, held up only by ground contact, with contact points at heel and toe. A powered ankle hinge (-20° to $+20^\circ$, 150 N·m maximum) is **the only actuated joint**. Above it, a sliding joint with a passive spring and damper represents shaft and socket flexibility (non-actuated). A 70 kg point mass on a 0.85 m leg represents the user.

The ground: MuJoCo's built-in contact solver is disabled; we supply our own contact force (F) at each point instead, based on sinking depth d and sinking speed $\dot{d}$:

$$F = k(d)d + c\dot{d} ; F \geq 0$$

Stiffness grows with depth ($k(d)$) to mimic how sand and mud firm up when you sink into it.

$$k(d) = k_0(1 + \alpha \cdot d/d_{\text{ref}})$$

where k_0 is the ground's baseline (undeformed) stiffness, α controls how fast it stiffens with depth, and d_{ref} is a reference depth that sets the scale of that stiffening. Past a preset yield force, the surface stays permanently dented.

The heel and toe portions of the foot model draw their properties independently, so the foot can straddle two surfaces.

Grip is modeled with velocity-regularized Coulomb friction, using a tanh instead of a hard sign function to avoid solver-breaking discontinuity at zero velocity:

$$F_t = \mu F_n \tanh(v_t/v_\epsilon)$$

Because the surface is a flat plane contacted at two points, penetration depth is a simple subtraction and each point's dent depth is stored as a state variable, so no mesh or finite-element modelling is required. Parameter ranges come from published terrain and prosthetic foot compliance data, and the model is verified against closed-form results: with the motor disabled the foot must settle at mg/k , oscillate at $\sqrt{k/m}$, and stay dented after yielding.

The episode: Starts at heel strike with randomized forward speed (0.8–1.4 m/s), downward speed (0.10–0.50 m/s), ankle angle and leg tilt. Each episode lasts **100 steps** covering heel strike, weight acceptance and mid-stance. Episode ends if the leg tips past $\pm 15^\circ$ from vertical, or the body drops below 70% of standing height.

Randomization: Ground properties are randomized per episode. A separate range is reserved for testing only, so the agent cannot succeed by memorizing surfaces.

RL Cast

Agent: In training it tries combinations of target angle, stiffness and damping across thousands of randomized surfaces, scored on falls and impact force. At deployment the weights are frozen: nothing reveals the ground before contact, so it lands with the best setting on average, then reads the force response and converges on the stiffness that suited similar surfaces. It adapts within the step but does not learn during it.

State (11 values per step): Ankle angle and angular velocity (encoder); ankle torque (motor current); vertical ground reaction force (pylon load cell); pylon compression; shank tilt angle and tilt rate (IMU); shank vertical acceleration; previous action (3). **The true ground parameters (stiffness, damping and friction) are never given to the agent, and it must infer them from the signals above.**

Action - The RL agent outputs a three-dimensional continuous action at each control step (200 Hz):

$$a = [\theta_d, K, B]$$

Desired ankle angle

$$\theta_d \in [-, 20^\circ, 20^\circ]$$

Ankle stiffness

$$K \in [5, 500] \text{ N m/rad}$$

Ankle damping

$$B \in [0.5, 20] \text{ N m s/rad}$$

Motor torque limit

$$\tau \in [-, 150, 150] \text{ N m}$$

Impedance control law

$$\tau = K(\theta_d - \theta) - B\dot{\theta}$$

where,

θ_d = desired ankle angle

θ = actual ankle angle

$\dot{\theta}$ = ankle angular velocity

Reward Function - The total reward consists of three terms:

$$r = r_{\text{survival}} + r_{\text{impact}} + r_{\text{smooth}}$$

2. Impact Penalty

1. Survival Reward

$$r_{\text{survival}} = +1$$

+1 reward for every step without falling.
If the agent falls, the episode terminates.

$$r_{\text{impact}} = -w_1 \left[\max \left(0, \frac{F}{mg} - \lambda \right) \right]^2$$

This penalizes excessive ground reaction forces above the selected threshold.

3. Smoothness Penalty

$$r_{\text{smooth}} = -w_2 \|a - a_{\text{prev}}\|^2$$

This penalizes large changes in desired ankle angle, stiffness, and damping between consecutive actions, encouraging smooth and realistic control.

Neural Network Structure

Two branches. Recent history tells the agent what the ground is, while the current reading tells it what the leg is doing right now.

The convolutional branch treats the **10 × 11** history as 11 sensor signals sampled at 10 points in time. **Two 1D convolution layers** scan along the time axis to pick out how each signal changes over the window. This captures cues such as how fast the ground force rises, which we expect to differ between firm and soft ground.

The MLP branch is a small fully connected network over the current step alone, so the present state is available directly rather than having to be recovered from history. The two outputs are joined and passed through two shared **256-unit layers (tanh activation)** into two heads. The Actor produces the three action values as the means of **three independent Gaussians**. Each mean is squashed and rescaled to its own physical range, rather than being used as a raw, unbounded output. The Critic estimates how good the current state is; it is used only to guide learning and is discarded at deployment.

A plain MLP over the flattened history is implemented first as a reference architecture. Both the CNN and MLP versions are evaluated under the full history-length ablation (10, 3, and 1 stacked steps), so the architecture comparison and the history-length comparison are reported together rather than as two separate sets of experiments.

Algorithm: PPO (Proximal Policy Optimization). It is an **actor-critic method**, matching the network above, which suits our continuous three-value action space. PPO is *on-policy*. Since our simulation is cheap (5 DOF, 100-step episodes), generating data is not the bottleneck. Furthermore, by limiting how far the policy can change in a single step, it makes training tolerant of an imperfect reward function. This algorithm will be compared against a **fixed-gain FSM impedance controller**.

Each of the following metrics will be averaged over at least three training runs started from different random seeds and reported as mean ± standard deviation, since any single run can succeed or fail by chance.

1. Peak ground reaction force (in body weights) against ground stiffness
2. Fall rate, split by failure type: tipping over versus collapsing
3. Generalization: metrics 1 and 2 measured on training ground versus testing ground.
4. History length: the same experiment repeated with 10, 3 and 1 stacked steps. If shortening the history makes performance worse, the agent is genuinely reading the ground from the force pattern rather than ignoring it

We used Claude Opus 5 (Anthropic) to evaluate ideas, summarize research papers that were used to justify the proposal and to simplify technical concepts.
We are completely responsible for the content and quality of the submitted work.